

RAP 課程延伸篇 10：用 RAP Action 重用既有 Classic REST 介面邏輯 (Legacy System Integration)

Lecture

這一課要回答什麼問題

rap01 ~ rap09 已經正式結案 (2026-08-17)。這一課是使用者事後提出的一個真實架構問題引發的延伸篇：**Classic REST (CL_REST_HTTP_HANDLER/CL_REST_RESOURCE)** 可以自訂 **Resource Class** 接收任意結構的 **JSON**、解析後寫進 **SAP** 表——這種「自訂介面 **JSON** 結構」的彈性，換成 **RAP** 架構要怎麼做到？RAP 的 **CRUD** 全部由框架處理，看起來沒有「自己寫一個 **Resource Class** 解析 **JSON**」的空間。

這一課用系統上一支真實在跑的正式生產介面當案例：**ZCL_QM_05_REST_HANDLER / ZCL_QM_05_RESOURCE / ZCL_QM_05_SERVICE** (套件 **ZQM1**，**QM005 WHMS** 介面——倉儲手持設備 / 主機系統把檢驗批使用決定回拋給 **SAP QM**)。這三個物件全程只讀取、不修改——這是這一課最重要的前提：不能因為要示範 RAP 就去動一支正在生產環境運作的介面。

先讀懂既有的 **Classic REST** 架構

```
ZCL_QM_05_REST_HANDLER (CL_REST_HTTP_HANDLER)
└ GET_ROOT_HANDLER 直接回傳 ZCL_QM_05_RESOURCE 實例 ( 單一 POST 端點，不需要 CL_REST_ROUTER )

ZCL_QM_05_RESOURCE (CL_REST_RESOURCE)
└ IF_REST_RESOURCE~POST :
    1. io_entity->get_string_data( ) 拿原始 JSON 字串
    2. /ui2/cl_json=>deserialize(...) 反序列化成 ZSQM005_INBOUND
    3. 呼叫 ZCL_QM_05_SERVICE=>process_requests( ... ) ← 純 HTTP I/O 跟業務邏輯在這裡切開
    4. /ui2/cl_json=>serialize(...) 序列化回應

ZCL_QM_05_SERVICE ( 純業務邏輯，不碰 HTTP 也不碰 JSON )
└ process_requests( IMPORTING it_request TYPE ztt_qm005_request
                    EXPORTING et_response TYPE ztt_qm005_outbound )
    1. 六條檢查 ( RT 單存在 / 全部 Pending / 同一個 UD 群組 / 明細存在 / 數量吻合 / 過帳類型吻合 )
    2. 呼叫 ZCL_QM_INSP_UD_POST=>process_rt_document ( 另一支共用過帳類別，QM002 )
    3. 寫入稽核 Log ( ZTQM01 )
```

這個架構本身已經是教科書等級的責任切分：**RESOURCE** 只管 HTTP / JSON I/O，**SERVICE** 是完全獨立、可單元測試、不依賴 HTTP 環境的純業務邏輯類別。這一課的整個重點就是：這種「業務邏輯類別已經跟 **I/O** 層分開」的架構，正好是能無痛套進 **RAP** 的前提——因為 RAP 真正要接手的只有 **RESOURCE** 那一層 (介面/資料落地)，**SERVICE** 那一層完全不用碰、也不應該碰。

資料型別本身就是巢狀的 (Header + Detail Array)

```

" ZSQM005_INBOUND
{ request : ZTT_QM005_REQUEST }          " 一次可以送多張 RT 單

" ZSQM005_REQUEST ( 單張 RT 單 )
{ header : ZSQM005_INB_H,                " 抬頭
  detail : ZTT_QM005_INB_D }              " 明細陣列 ( 可以是空的，代表整張 RT 單下全部檢驗批
一起過帳 )

```

這正是本課程 rap08 教過的「Header + Item」結構，只是這裡不是要用 RAP Composition 塑模一個新的持久化 Entity，而是既有的 **Classic DDIC 巢狀結構** (`define structure`，純 ABAP Structure，`detail` 欄位本身是一個 Table Type) ——這種「結構裡包一個 Table 欄位」的寫法，一般 ABAP 早就支援，跟 RAP 完全無關。

第一次嘗試：Abstract Entity + Deep Action (SAP 官方文件的建議做法，但這系統不支援)

網路上 / SAP 官方文件建議：巢狀輸入用 **CDS Abstract Entity** (`define abstract entity`，不落地資料庫、純型別載體) + **Deep Action** (`action X deep table parameter <AbstractBDEF>;`) 塑模。這一課出題前照官方文件的做法完整試了一輪，過程與結論已經寫進 [rap01_why_rap.md](#) 「Deep Parameter 版本演進歷程」那一節，這裡只重述結論：

- CDS 層 (`define abstract entity + composition/association to parent`) 在這系統可以編譯、啟用——比預期寬鬆
- 但 Abstract BDEF 的 `with hierarchy` 這一行編譯錯誤：`"BOPF | draft" expected, not "hierarchy"`.
- Action 定義的 `deep / deep table` 關鍵字，剖析器直接不認得：`"; | external | parameter | result" expected, not "deep"`.
- 查官方 `ABENRAP_FEATURE_TABLE`：這整組語法 (`abstract / with hierarchy / deep parameter / deep mapping`) 全部要 On-Premise **7.56 (≈S/4HANA 2021)** 才有，這系統是 **7.54 (1909)**，差了兩個年度版本

驗證用的失敗案例 (`ZA_RAP10_HDR / ZA_RAP10_DTL / ZA_RAP10_HDR.bdef.abap`) 保留在 `src/` 當反面教材，檔案開頭的註解完整記錄了兩種失敗寫法跟對應的錯誤訊息。

第二次嘗試：扁平 Parameter 直接引用既有 DDIC 巢狀型別 (這一課實際採用的做法)

官方文件其實明講：Action 的 (非 Deep) 扁平 `parameter` 子句，型別可以是「**CDS Abstract Entity**，或者一個 **classic DDIC type**」——後者完全沒有版本限制 (`parameter` 子句本身從 Unmanaged 7.53 / Managed 7.54 就有)。既然 `ZTT_QM005_REQUEST` 本身就已經是一個現成的、巢狀的 classic DDIC Table Type，直接拿來當 Action 的扁平參數：

```

static action SubmitWhmsRequest parameter ZTT_QM005_REQUEST result [1..*]
ZSQM005_OUTBOUND;

```

實測一次就編譯乾淨過關 (無任何錯誤/警告)。這代表：只要巢狀資料結構已經用傳統 **ABAP Structure/Table Type** 定義好 (不透過 RAP 自己的 **Composition** 機制)，Action 完全可以用「一個參數扛住整包深層資料」，不需要 Deep Parameter 這整組 7.56+ 才有的機制。差別只在於拿不到 RAP 原生的 `%control` / 深層 `%target` 這類 EML 語法糖，但對「把資料整包塞進 Action、內部邏輯自己解析」這種情境完

全夠用——而且更貼近原始需求：我們本來就不是要把 WHMS 的 Detail 陣列塑模成一個獨立的、有自己生命週期的 RAP Composition 子實體，只是要把一包既有格式的資料原封不動送進既有的業務邏輯。

最終架構：Adapter 層 vs. BO 層，Action 呼叫既有 Service Class

```
( 未來若要串真的 WHMS ) 自訂 Wire Adapter ( 可以是 Classic REST / IDoc / Proxy，不受 RAP 限制 )
| 解析任意格式的 legacy JSON/XML，轉換成 ZTT_QM005_REQUEST 這個既有型別
▼
EML: MODIFY ENTITIES OF zi_rap10_log ENTITY Log
      EXECUTE SubmitWhmsRequest FROM VALUE #( ( %cid = 'C1' %param = lt_request ) )
      RESULT ... FAILED ... REPORTED ....
|
▼
RAP Action Handler ( Unmanaged，FOR MODIFY )
| 逐一 %param 呼叫既有邏輯，零修改、零重複：
▼
ZCL_QM_05_SERVICE=>process_requests( ... )    ← 完全不動，Classic REST 介面現在還在呼叫同一支
|
▼
INSERT ZRAP10_LOG ( 這一課專用的訓練稽核表，不寫真正的 ZTQM01 )
```

這個架構回答了原本的問題：Classic REST 的「自訂 JSON 結構」彈性，被搬到「Adapter 層」——那一層完全不受 RAP 限制，可以是任何協定（本課程沒有另外建一個新的 REST Resource，直接用 EML 呼叫端程式模擬 Adapter 的角色，因為重點是驗證 RAP Action 這一段）。RAP 真正接手的只有「呼叫既有業務邏輯 + 落地稽核記錄」這一段，跟 Classic REST 的 **RESOURCE** 層原本負責的事完全對應，只是換了一個呼叫入口。

ZCL_QM_05_SERVICE 這支類別本身完全沒有被修改過一行——這是重用，不是重寫。

⚠️ ⚠️ 實測結論：這個 Action 現在的參數形狀，OData V2 曝露不出來——不是風格選擇，是唯一可行的路

上面的架構圖畫了「未來若要串真的 WHMS」的 Adapter 層，但一開始沒有立刻確認：這個 **RAP Action** 本身，能不能像一般 **RAP BO** 那樣，透過 **Service Definition / Service Binding** 直接曝露成 **Postman** 打得到的 **OData** 端點？如果可以，Adapter 層某種程度上就是可省的（讓 legacy 系統直接打 OData）。實測結果：不行，而且原因比想像中隱蔽。

建立 **ZRAP10_SD** (**expose ZI_RAP10_LOG as Log;**)，使用者在 Eclipse 建 **ZRAP10_SB** (OData V2 - UI) 並成功 Publish (**Local Service Endpoint: Published**)。Fiori Elements Preview 也正常開得出畫面、抓得到 3 筆稽核記錄——這一步證實 **Publish** 本身沒問題，**Entity** 的標準 **CRUD** 曝露完全正常。

但直接看 **\$metadata** ([https://<host>/sap/opu/odata/sap/ZRAP10_SB/\\$metadata](https://<host>/sap/opu/odata/sap/ZRAP10_SB/$metadata)) 才發現真正的問題。OData V2 沒有原生的「Action」概念，RAP Action 對應到 **FunctionImport**：

```
<FunctionImport Name="SubmitWhmsRequest"
  ReturnType="Collection(cds_zrap10_sd.Zsqm005Outbound)" m:HttpMethod="POST"/>
```

這個標籤是自我封閉的，完全沒有任何 `<Parameter>` 子元素。Gateway 在產生 Metadata 時，因為 `ZTT_QM005_REQUEST` 這個「Table 包 Header 結構 + Detail Table」的巢狀型別不符合 OData V2 對 Action 輸入參數的限制（官方文件明講：「For importing parameters only simple types are supported」——這條限制寫在最新版官方文件裡，不是這系統版本特有的，換到更新的系統大概率一樣），**Gateway** 選擇把整個參數定義默數丟掉，而不是讓 **Publish** 失敗。`SubmitWhmsRequest` 這個名字還在 Metadata 裡（容易誤以為「有曝露成功」），但完全沒有任何管道可以把 Header/Detail 資料傳進去——對照組是 Return Type（`ZSQM005_OUTBOUND`，扁平結構、沒有巢狀 Table）正常變成 `ComplexType` 保留下來，證實問題精確出在「結構裡包 Table」這件事本身，不是整個 Action 機制。

這是比「完全曝露失敗」更值得記住的失敗模式：Publish 成功、Action 名稱看得到，很容易讓人誤判「應該能用」，但實際上少了參數就等於這個端點沒有意義。這一發現把「**Adapter** 層留在 **RAP** 外面」從架構建議，升級成這個資料形狀在這系統上的唯一可行做法——不管願不願意，這個 Action 天生就只服務 EML / 內部 ABAP 呼叫端。

真的建一個 Wire Adapter，端到端證明整條路

架構圖畫完、上面那個限制也確認了，接下來實際建一個 Adapter，把「JSON 字串進、JSON 字串出」整條路徑跑一次，不透過真的 HTTP/SICF（那部分是 GUI 設定，不影響這裡要驗證的架構本身）：

- `ZCL_RAP10_REST_HANDLER ( CL_REST_HTTP_HANDLER ) / ZCL_RAP10_RESOURCE ( CL_REST_RESOURCE )`——結構完全比照 production 的 `ZCL_QM_05_REST_HANDLER / ZCL_QM_05_RESOURCE`，**JSON** 契約沿用同一個 `ZSQM005_INBOUND`，`IF_REST_RESOURCE~POST` 唯一的差異是不直接呼叫 `ZCL_QM_05_SERVICE`，改呼叫 `HANDLE_REQUEST`（EML 呼叫 `SubmitWhmsRequest`）
- 為了不用真的架 HTTP 端點就能驗證，`HANDLE_REQUEST` 被設計成一個可以直接呼叫的 Class Method：`IMPORTING iv_json_in TYPE string RETURNING VALUE(rv_json_out) TYPE string`，`POST` 方法本身只是薄薄一層 I/O 包裝（跟 rap10 一路示範的「業務邏輯跟 I/O 分開」原則一致）
- `ZR_RAP10_ADAPTER_DEMO` 用 `programrun` 直接餵一段跟 Postman 會送的 HTTP Body 一模一樣的原始 JSON 字串進去，驗證完整路徑：**JSON 字串** → **Adapter 解析** → **EML 呼叫 RAP Action** → **Action 呼叫未修改的 `ZCL_QM_05_SERVICE`** → 錯誤訊息回傳 → 序列化回 **JSON 字串**

```
=== JSON IN ===
{"REQUEST":[{"HEADER":
{"ZWHMS_NO":"RAP10ADAPT","ZRT_NO":"9999999999","BUDAT":"20260821","BLDAT":"20260821","
ZTRAN_TYPE":"1","BKTXT":"Adapter demo"},"DETAIL":[]}]
=== JSON OUT ===
{"RESPONSE":[{"ZWHMS_NO":"RAP10ADAPT","ZRT_NO":"9999999999","MSGTY":"E","MESSAGE":"RT
單不存在於系統"}]}
```

跟 Classic REST 介面收到同樣輸入會回應的內容完全一致，證實 Adapter 換了呼叫入口（EML 而非直接呼叫 Service Class），但業務行為完全沒變。

附帶一個小發現，一併誠實記錄：驗證程式額外測了一段故意壞掉的 JSON（`{ this is not valid json }`），預期會走 `CATCH cx_root` 回報「Invalid JSON request body」，但實際輸出是 `{}`（2 個字元）——`/ui2/cl_json=>deserialize` 對這個特定的壞字串並沒有拋例外，是安靜地留下空結構繼續往下執行，導致 EML 帶著空的 Request Table 執行、回傳空陣列、`compress` 模式把空陣列從輸出裡省略掉。這代表 `/ui2/cl_json` 的容錯行為不能完全信賴「格式錯就一定拋例外」，如果真的要接生產流量，Adapter 層的輸入驗證需要更嚴謹（例如額外檢查關鍵欄位是否真的有值），不能只依賴 `deserialize` 的例外機制。

✅✅ 真的用 Postman 打通了：SICF 掛載 + 端到端驗收

programrun 驗證的是「JSON 字串直接餵給 Class Method」，跳過了真正的 HTTP / SICF 那一層。這一課最後一步是把這層也補上，變成一個真正能被 Postman 呼叫的端點。SICF 節點掛載沒有 ADT API (跟 REST 課程 rs01 同一個已知限制)，使用者在 SAP GUI 操作：

1. SICF → Hierarchy Type Service，展開到 default_host/sap/bc
2. bc 右鍵 → New Sub-Element → zrap10 (純目錄節點，不用掛 Handler / 不用 Activate)
3. zrap10 右鍵 → New Sub-Element → whms
4. whms 節點 → Handler List 頁籤 → 新增 ZCL_RAP10_REST_HANDLER
5. 右鍵 whms → Activate Service

測試網址 (外網對外別名，不要用 SICF 「Test Service」按鈕開出的內網網址，這是 REST 課程 rs01 已經記錄過的坑)：

```
POST https://erpdemo01.itts.com.tw:44300/sap/bc/zrap10/whms?sap-client=130
```

因為 HANDLE_CSRF_TOKEN 是空實作 (比照 production 的 Server-to-Server / Basic Auth 設計)，Postman 不需要先抓 CSRF Token，直接 POST 就成功。使用者實測結果：200 OK，368 ms，回應內容跟 programrun 驗證過的完全一致：

```
{"RESPONSE": [{"ZWHMS_NO": "POSTMANTEST", "ZRT_NO": "9999999999", "MSGTY": "E", "MESSAGE": "RT  
單不存在於系統"}]}
```

這是整堂課的收尾證據：一個真正的 HTTP POST、真正的 JSON Body、真正的 SICF 路由，跟一路用 programrun 無頭驗證的結果分毫不差——證實從架構圖到可執行程式碼，每一層的推論都站得住腳。

RAP + Adapter 跟純 Classic REST，差異到底在哪裡？(不是 Wire 層程式碼，是背後接的東西)

Postman 打通之後很自然會問：ZCL_RAP10_RESOURCE 跟 production 的 ZCL_QM_05_RESOURCE，程式碼結構幾乎一模一樣，都是 CL_REST_RESOURCE 子類、都覆寫 POST——這樣做到底差在哪？答案不在 Wire 層 (那一層本來就該長一樣，見上面「先讀懂既有的 Classic REST 架構」)，差異在這個 Resource 呼叫的下一步是什麼、以及那個下一步還能被誰重用：

	Classic REST (production ZCL_QM_05_*)	RAP + Adapter (這一課 ZCL_RAP10_*)
Wire 層 程式碼	CL_REST_RESOURCE 子類，POST 方法	完全一樣，CL_REST_RESOURCE 子類， POST 方法
呼叫下 一步的 方式	直接呼叫 ZCL_QM_05_SERVICE=>process_requests() (普通 Method 呼叫)	MODIFY ENTITIES ... EXECUTE SubmitWhmsRequest ... (EML，RAP 標 準協定)

Classic REST (production ZCL_QM_05_*)		RAP + Adapter (這一課 ZCL_RAP10_*)
這個「下一步」還能被誰呼叫	只有寫 ABAP 程式、明確 IMPORT 這個類別的人——沒有第二個管道	任何懂 EML 的呼叫端：別的 RAP BO 的 Determination / Validation / Action、別支報表、(如果參數形狀允許) OData / Fiori——不用重寫，直接用同一句 EML 語法接進來
讀取 / 稽核那一側	ZTQM01 稽核表要另外寫報表/查詢程式才能看	ZI_RAP10_LOG 建好當下就能被 Open SQL / RAP READ / OData V2 直接查詢 (已用 Fiori Preview 驗證過)，完全不用額外寫程式
交易/鎖定語意	自己在 Service Class 裡處理	框架標準化 (lock master 、Transactional Buffer、 COMMIT ENTITIES)，不用自己重新發明

一句話講差異：不是「RAP 取代了 Wire Adapter」，是「**Wire Adapter** 背後接的東西，從一支只能被寫死呼叫的類別，換成一個誰都能用標準協定 (**EML**) 接進來的 **RAP BO**」。今天想要「連動觸發 WHMS 過帳邏輯」，Classic REST 版本只能直接 import **ZCL_QM_05_SERVICE** (繞過所有治理)；RAP 版本可以用一句 **EXECUTE SubmitWhmsRequest** 接進來，不用知道底層實作細節。

誠實補充一個限制，這個案例剛好踩到：RAP 真正吸引人的地方通常是「同一份邏輯，OData / Fiori 也能免費用」——但前面已經證實這個 **Action** 因為巢狀參數，**OData V2** 曝露不出來，所以「Fiori App 也能直接呼叫這個 Action」這個好處，在這個具體案例上並沒有真的兌現。如果 Action 的參數是扁平 / 簡單型別，RAP 這條路換來的好處會更明顯 (同時拿到 EML + OData 兩個管道)；像這一課這種巢狀參數的案例，RAP 帶來的實質好處主要是讀取那一側 (稽核 **Log** 免費可查) 跟未來被其他 **RAP BO** 用 **EML** 重用的可能性，Action 本身不能被 Fiori 直接呼叫這件事目前是拿不到的。

為什麼用一個保證不存在的 RT 單號測試

ZCL_QM_05_SERVICE=>process_requests 內部會查真實檢驗批 (**get_lots_for_rt**) 並在檢查全過時呼叫真正的過帳邏輯 (**ZCL_QM_INSP_UD_POST=>process_rt_document**，會建立真實的物料憑證)。這一課的驗證程式故意用一個保證不存在的 RT 單號 (**9999999999**)——**check_rt_exists** 會查到 0 筆、直接進入第一條檢查失敗的錯誤路徑 (**RT單不存在於系統**)，完全不會走到過帳那一段，零風險驗證「RAP Action 呼叫到的是同一套業務邏輯、同一組錯誤訊息」，不需要冒真的觸發過帳的風險。

✅✅ Cloud 對照組：Deep Table Parameter 的版本落差是真的，但曝露限制比想像中更根本

上面「第一次嘗試」踩到的 **with hierarchy / deep [table] parameter** 語法拒絕，查 **ABENRAP_FEATURE_TABLE** 顯示這組語法要 On-Premise **7.56 (≈2021)** 才有，這套地端系統是 7.54 (1909)，差兩版。這段用真正的 **ABAP Cloud** (BTP Trial，**ZRAPCLOUD** 套件) 驗證這個版本落差判斷是否成立，順便驗證「OData V4 是不是就能曝露巢狀 Action 參數」這個常見猜測。

✅ **Test A** : **with hierarchy** 在 Cloud 上編譯、啟用成功——建立 **ZA_RC10_HDR (abstract; strict(2); with hierarchy; + composition [0..*] of ZA_RC10_DTL) / ZA_RC10_DTL (association to parent)**，跟地端「**"BOPF | draft" expected, not "hierarchy"**」的硬性拒絕完全相反，Cloud 上乾淨過關。兩個 Abstract Entity 互相參照 (Composition ↔ to-parent association) 第一次批次啟用會互卡，解法是

先拆開單獨啟用、再補上關聯重新啟用。證實版本落差判斷成立：這組語法確實是地端系統版本太舊才用不了，不是永久性限制。

✅ **Test B**：建一個真正持久化的 **RAP BO**，掛 **deep table parameter Action**，端到端驗證邏輯正確：

- **ZRC10_ROOT** (Table) / **ZI_RC10_ROOT** (CDS Root View Entity) / **BDEF** (Managed , **static action submit deep table parameter ZA_RC10_HDR result [1..*] \$self;**) / **ZBP_I_RC10_ROOT** (實作類別) 全部建立並啟用成功，Cloud 上完全支援這組語法。
- ⚠️ **Eclipse 操作細節**：**BDEF** 沒有先 **Activate**，**Ctrl+1**「**Create behavior implementation class**」快速修正找不到目標——這系統的 **Ctrl+1** 骨架生成是讀「已啟用」的 Behavior Definition 反查方法簽章，**BDEF** 還是 **Inactive** 狀態時 **Eclipse** 認不出要生成什麼。正確順序：先 **Activate BDEF**，確認成功後才對類別名稱按 **Ctrl+1**。這是本課程第一次在 Cloud 連線上遇到、地端從未觀察到的細節（地端 **Unmanaged** 實作類別大多是 Claude 用 ADT API 直接建立，沒機會踩到這個情境）。
- ⚠️ **RESULT 衍生型別的坑**：**result [1..*] \$self** 這種「非 Factory Action 但自己 CREATE 新實例」的寫法，**TYPE TABLE FOR ACTION RESULT** 官方文件列出 **%key / %tky / %pky / %param / %cid** 都是候選元件，但這個案例只有 **%cid + %param** 真正存在（**%param** 本身就是完整實體資料，含 Key 欄位，不需要另包 Key 結構）——逐一單獨測試 + 用 **abap_activate** 的後端編譯錯誤（比 IDE 端 **get_abap_diagnostics** 可靠，後者在依賴的 **BDEF** 曾經語法錯誤時回報過一次不同步的「0 errors」假象）才確認下來。
- **ABAP Unit Test 驗證通過**（**ZCL_RC10_ACTIONTEST**，這個連線沒有像地端 **programrun** 的無頭執行工具，改用 **ABAP Unit** 斷言）：一次 **EML** 傳 2 個 Header（各帶 2 / 1 筆 Detail），**submit** 正確建立 2 筆 Root 實例，**descr** 欄位正確編碼巢狀 Detail 筆數——證實巢狀資料真的能從 **EML** 呼叫端傳遞、逐層攤平讀取，不只是編譯過關：

`` [PASS] LTC\_SUBMIT\_ACTION [PASS] DEEP\_TABLE\_PARAMETER\_WORKS (0.230s) ``

⚠️ ⚠️ 決定性負面結果：**deep table parameter** 完全不能透過 **OData** 曝露，不分 **V2 / V4**——建 Service Binding（不管 MCP 工具走 **OData V2**、還是 **Eclipse** 精靈明確選 **OData V4 - UI**）在編譯階段就一律被擋下：

```
Error during compilation. See next line for details.  
Error in entity 'ZI_RC10_ROOT(CDS)': Action SUBMIT: Deep table parameters are not supported for OData exposure
```

這跟地端「**OData V2** 曝露時參數被默默丟掉、**Publish** 還能成功」（本課前段「⚠️ ⚠️ 實測結論」那一節）性質不同：地端是曝露成功但參數消失（診斷困難：要挖 **\$metadata** 才發現），Cloud 是編譯期直接拒絕（診斷容易：錯誤訊息直接講明白）。但兩者殊途同歸——結論一致：**deep table parameter** 從設計上就是給 **ABAP** 端 **EML** 消費者用的機制，不是給 **OData / Fiori** 這種外部協定消費者用的，跟 **OData** 版本或系統版本都無關，是 **RAP** 框架的設計邊界。

這個結論回頭印證了這一課主架構的選擇是對的，不是將就：正文一路用「扁平 Parameter 直接引用既有 **DDIC** 巢狀型別」+「Adapter 層（**ZCL_RAP10_RESOURCE**）自己解析 **JSON** 再用 **EML** 呼叫」這個做法，不是因為地端系統版本舊才退而求其次——即使換到版本更新、語法更完整的 **Cloud** 系統，**deep table parameter Action** 一樣沒辦法被 **Fiori / Postman** 直接打到，**Wire Adapter** 這一層本來就是任何想要「巢狀 **JSON** 進、**RAP** Action 接」這種需求的必經之路，跟系統版本無關。

✅✅ **Cloud** 上的 **Wire Adapter** 該用什麼：**Classic REST** 直接被拒絕，**HTTP Service** 才是正解

上一節證實 Wire Adapter 這一層在 Cloud 上也是必經之路，但不能照搬地端這一課用的 **Classic REST** (**CL_REST_HTTP_HANDLER** / **CL_REST_RESOURCE**) ——這兩個類別雖然在 Cloud 系統的 Repository 裡看得到 (因為底層 Kernel / **SAP_BASIS** 是共用的)，但沒有被列進「Released API」清單，Cloud 的受限語言版本直接拒絕引用：

```
DATA probe TYPE REF TO cl_rest_http_handler.
```

Activation FAILED:

The use of Class CL_REST_HTTP_HANDLER is not permitted.

(這是在 **ZCL_RC10_ACTIONTEST** 裡臨時加一行探測、確認錯誤訊息後就還原的實測，不是查文件猜的。)

Cloud 原生的對應機制是 **HTTP Service** (**IF_HTTP_SERVICE_EXTENSION**) ——這是 ABAP Cloud 正式引入的 Released Repository Object，在 Eclipse ADT 用「New → HTTP Service」精靈建立，角色跟 Classic REST 完全對應 (能讀原始 Body、自己解析 JSON、自己組回應)，但不需要 SICF 手動掛節點——底層還是走同一套 ICF / HTTP 路由 (Publish 之後 Project Explorer 的 **Others** → **SICF** 底下真的會多一個自動產生的節點，證實 SICF 機制沒有消失，只是 Cloud 開發者不用手動碰它)。

用這個機制重做了一次 rap10 的 **Wire Adapter** (**ZRC10_HTTP** / **ZCL_RC10_HTTP**)，端到端驗證分兩層：

1. **ABAP / EML 層**——完整驗證成功：**HANDLE_REQUEST** 用 Cloud Released 的 JSON API (**xco_cp_json=>data->from_string(...)->write_to(...)**) 解析巢狀 JSON (Header 陣列，每筆帶巢狀 Detail 陣列)，轉換成 **TYPE TABLE FOR HIERARCHY za_rc10_hdr** 後直接呼叫 **MODIFY ENTITIES ... EXECUTE submit**——因為 **HTTP Service** 完全不經過 **OData** 協定層，前一節「**Deep Table Parameter** 不支援 **OData** 曝露」這個限制在這裡完全不適用。改用 ABAP Unit Test (**ZCL_RC10_ACTIONTEST**，因為這個連線沒有地端 **programrun** 那種無頭執行工具) 驗證，一次傳 2 個 Header (各帶 2 / 1 筆 Detail) 呼叫 **submit**，斷言全部通過：

`` **[PASS] LTC\_SUBMIT\_ACTION [PASS] DEEP\_TABLE\_PARAMETER\_WORKS (0.230s)** `` 這證實：巢狀 **JSON** 進、**RAP Deep Table Parameter Action** 接，這條路在 **Cloud** 上完全可行——只要不透過 **OData**，改用 **HTTP Service** 直接用 **ABAP** 程式碼呼叫 **EML** 就行。

1. 外部 **HTTP** 呼叫層——部分驗證，卡在 **Postman** 認證機制，不是設計問題：
 - **ZRC10_HTTP** 用 Eclipse 的「Publish Locally」按鈕發布成功 (第一次遇到暫時性 500，重試就過)，Project Explorer 確認自動生成 SICF 節點
 - 用瀏覽器直接點 **HTTP Service** 編輯畫面裡的 **URL** 連結 (沿用 Eclipse 登入的瀏覽器 Session) ——回應是 **500 Internal Server Error** (**An exception has occurred that was not caught**)，不是 **401/403**：這證實認證與授權都過關、請求真的打進了 **HANDLE_REQUEST**，500 只是因為瀏覽器點連結送的是不帶 Body 的 GET，程式碼對空字串做 JSON 解析時未捕捉例外而已 (程式碼沒有防禦 GET / 空 Body 的情況，不影響「巢狀 JSON POST 邏輯本身是對的」這個結論)
 - 改用 **Postman** 送真正帶 **JSON Body** 的 **POST**，用 **OAuth 2.0 Password Grant Type** 拿 **Bearer Token** 呼叫，得到 **401 Unauthorized**——一開始懷疑跟 **[[fiori-elements-tooling]]** 課程記錄的「共用 Trial 帳號沒有 **SAP_BR_ADMINISTRATOR**、無法走 IAM App / Business Catalog / Business Role 這條鏈」是同一個限制，但瀏覽器測試已經推翻這個假設——瀏覽器 Session 完全沒有經過 IAM App 這條鏈就成功打進程式碼，證實 Locally Published 的 **HTTP Service** 對開發者自己不需要這整套管理員層級授權。真正的原因範圍縮小到：**Postman** 的 **OAuth Bearer Token** 認證方式，

跟這個端點期待的 (瀏覽器 **SSO Session**) 認證方式對不上，可能是 Token 的 audience / scope 不匹配，或 Locally Published 端點設計上本來就只吃互動式瀏覽器 Session、不吃純 Bearer Token——沒有進一步深究 (下一步是把瀏覽器 Cookie 複製到 Postman 改用 Session-based 認證，判斷投入產出比後決定不繼續，如實記錄現有證據)

結論：這一課如果要在真正的 ABAP Cloud 系統上重做，Wire Adapter 那一層答案很明確——用 **HTTP Service**，不是 **Classic REST** (後者編譯不過)；**deep table parameter** 這種巢狀 Action 參數在 ABAP / EML 層完全可行，只是不能透過 OData / Fiori 曝露，這點跟地端、跟 Cloud 都一樣；外部 HTTP 呼叫的最後一哩因為這個共用 Trial 環境的認證機制細節卡住，但 ABAP 端的邏輯正確性已經有 ABAP Unit Test 跟瀏覽器測試 (500 而非 401) 雙重佐證，不影響整體架構結論的可信度。

Eclipse ADT Step by Step (如果你想自己重建這個案例)

1. **Table**：對 **\$TMP** 套件右鍵 → New → Database Table，命名 **ZRAP10_LOG**，欄位比照 **zrap10_log.tabl.abap** (**mandt** / **log_id** (**SYSUID_X16**) / **zwhms_no** / **zrt_no** / **budat** / **bldat** / **ztran_type** / **bktxt** / **msgty** / **message** / **created_at**)
2. **CDS View**：對 Table 右鍵 → New → Data Definition，Templates 選「Define View (obsolete as of AS ABAP 7.57)」，命名 **ZI_RAP10_LOG**，內容比照 **zi_rap10_log.ddls.abap**
3. **BDEF**：對 CDS View 右鍵 → New → Behavior Definition，implementation type 選 **Unmanaged**，內容比照 **zi_rap10_log.bdef.abap**——注意 **parameter** 子句直接寫 **ZTT_QM005_REQUEST** (既有的 production Table Type，不用自己另外建型別)
4. **實作類別**：BDEF 建立時 Eclipse 會提示建立實作類別骨架 (**ZBP_I_RAP10_LOG**)，Local Types Include (Ctrl+點類別名或右鍵 Show Includes) 貼上 **zbp_i_rap10_log.clas.locals_imp.abap** 的內容
5. Ctrl+S 存檔 → Activate (Ctrl+F3，會一次啟用 Table / CDS View / BDEF / 實作類別)
6. 建驗證程式 **ZR_RAP10_DEMO** (Program)，內容比照快照，F8 執行看 Classical List 輸出

學習目標

- 能解釋「Wire Adapter 層」跟「RAP BO 層」的責任切分，知道為什麼 Classic REST 的自訂 JSON 彈性應該留在 Adapter 層，不是硬塞進 RAP 框架
- 能查證並解釋 Deep Parameter (Abstract Entity 當 Action 的巢狀輸入) 在特定系統版本上為什麼不可用，並知道去哪裡查 (**ABENRAP_FEATURE_TABLE**)
- 能用既有的 **classic DDIC** 巢狀型別 (不新建 CDS Abstract Entity) 直接當 RAP Action 的扁平 **parameter**，理解這是官方文件明講允許的合法用法，不是 **workaround**
- 能設計一個「Action 呼叫既有業務邏輯類別」的 Unmanaged BDEF，並解釋為什麼這樣做能達到「重用不重寫」 (既有 Classic REST 介面完全不用修改)
- 能設計一個零風險的驗證方式 (保證不存在的測試資料，走安全的錯誤路徑)，驗證「新入口跟舊入口共用同一套業務邏輯」而不冒觸發真實副作用 (過帳) 的風險
- 能讀懂 OData V2 **\$metadata** 裡的 **FunctionImport**，並判斷「Publish 成功、Action 名稱出現在 Metadata 裡」不等於「這個 Action 真的能被外部呼叫」——知道要進一步檢查有沒有 **<Parameter>** 子元素
- 能設計一個結構跟既有 Classic REST 介面平行對照的 Wire Adapter (同一份 JSON 契約，只換內部呼叫方式)，並用「直接呼叫核心方法、餵一段原始 JSON 字串」的方式無頭驗證，不需要真的架 HTTP 端點

物件清單

物件	名稱	型別	備註
訓練用稽核表	ZRAP10_LOG	TABL/DT	不是真正的 ZTQM01，訓練專用，不影響生產資料
CDS Interface View	ZI_RAP10_LOG	DDL/DF	
Behavior Definition	ZI_RAP10_LOG (BDEF)	BDEF/BDO	Unmanaged，唯一操作是 <code>static action SubmitWhmsRequest</code>
實作類別	ZBP_I_RAP10_LOG	CLAS/OC	Handler 呼叫既有 <code>ZCL_QM_05_SERVICE=>process_requests</code> (唯讀呼叫，未修改)
驗證程式	ZR_RAP10_DEMO	PROG/P	EML 呼叫， <code>programrun</code> 無頭驗證通過
反面教材 (Deep Action 失敗案例)	ZA_RAP10_HDR / ZA_RAP10_DTL	DDL/DF	CDS 層可編譯，BDEF 層不行，見檔案開頭註解
Service Definition	ZRAP10_SD	SRVD/SRV	<code>expose ZI_RAP10_LOG as Log;</code> ，用來驗證 OData V2 曝露限制
Service Binding	ZRAP10_SB	SRVB/SVB	使用者於 Eclipse 建立 + Publish， <code>\$metadata</code> 證實 <code>SubmitWhmsRequest</code> 的參數被 Gateway 丟棄 (見 Lecture)
Wire Adapter HTTP Handler	ZCL_RAP10_REST_HANDLER	CLAS/OC	結構比照 <code>production ZCL_QM_05_REST_HANDLER</code>
Wire Adapter Resource	ZCL_RAP10_RESOURCE	CLAS/OC	<code>HANDLE_REQUEST</code> 可獨立呼叫 (JSON 進 JSON 出)， <code>POST</code> 只是薄包裝，呼叫 RAP Action 而非直接呼叫 <code>ZCL_QM_05_SERVICE</code>
Adapter 驗證程式	ZR_RAP10_ADAPTER_DEMO	PROG/P	<code>programrun</code> 無頭驗證 JSON 字串完整路徑，含格式錯誤 JSON 的邊界案例

全部物件都在 `$TMP` 套件，已建立並啟用 (讀取 active 版本內容與寫入內容逐字相符)。這一課的物件由 **Claude** 建立 (延續 rap01 ~ rap03 的模式，因為這是使用者提出的架構討論延伸出的 R&D / 示範案例，不是課綱既定的動手練習課)。

唯讀引用、完全未修改的生產物件 (供對照，不在這一課的物件清單裡)：`ZCL_QM_05_REST_HANDLER` / `ZCL_QM_05_RESOURCE` / `ZCL_QM_05_SERVICE` / `ZCL_QM_INSP_UD_POST` / `ZCL_QM_AMDP_INSP_05` (套件 ZQM1)。

驗證方式

ZR_RAP10_DEMO 透過 programrun 無頭執行：

```
=== RAP10: Legacy WHMS integration via RAP Action + EML ===
Reusing production class ZCL_QM_05_SERVICE (package ZQM1) unchanged.
=== EML RESULT ===
RAP10DEM01          9999999999 E
RT單不存在於系統

=== ZI_RAP10_LOG content after this run (Open SQL) ===
RAP10DEM01          9999999999 E
RT單不存在於系統

=== Sanity check ===
MATCH: reused ZCL_QM_05_SERVICE validation fired correctly
via RAP Action - same error path/message text as the real
WHMS interface would produce, zero risk (fake RT number,
no real inspection lot touched, no real posting attempted).
```

RT單不存在於系統 是 ZQM05 Message Class 的真實生產訊息文字

(ZCL_QM_05_SERVICE=>c_msg_rt_not_exist)，透過 RAP Action 觸發，跟 Classic REST 介面收到同樣輸入會回應的錯誤訊息一字不差——證實 RAP Action 呼叫到的是同一支、完全沒有修改過的業務邏輯類別。

ZI_RAP10_LOG 之後也能被任何 RAP / OData Consumer 直接查詢（這是 Classic REST 原本沒有「免費」拿到的能力——ZTQM01 稽核表要另外自己寫查詢程式或報表才能看，ZI_RAP10_LOG 建好之後配上 Service Binding 就能直接給 Fiori App 用）。

思考題

1. 如果之後真的要接一個會呼叫這個 Action 的自訂 Wire Adapter（例如一個新的 Classic REST Resource Class），Adapter 要做的事只有「解析 JSON → 組出 ZTT_QM005_REQUEST → 呼叫 EML」，這跟原本 ZCL_QM_05_RESOURCE~POST 的程式碼結構有什麼不同？（提示：對照 ZCL_RAP10_RESOURCE~HANDLE_REQUEST 跟 production ZCL_QM_05_RESOURCE~POST，兩者只差第 3 步）
2. 這一課驗證程式故意用一個保證不存在的 RT 單號，如果哪天真的要驗證「過帳真的成功」這條路徑，你覺得除了「拿真實資料測」之外，有沒有更安全的做法？（提示：回顧 Enhancement 課程「先假資料、後真資料」的安全閘設計原則）
3. ZTT_QM005_REQUEST 這個型別本來就存在（ZCL_QM_05_SERVICE 的 it_request 參數型別），這一課完全沒有新建任何 CDS Abstract Entity 就達成了巢狀資料傳遞——如果哪天系統升級到支援 Deep Parameter 的版本（7.56+），你覺得還有沒有理由要改用 Abstract Entity + Deep Action？兩種做法的取舍是什麼？
4. \$metadata 證實 OData V2 把 SubmitWhmsRequest 的參數默默丟掉，而不是讓 Publish 直接失敗——如果你是負責維護這個系統的人，這種「看起來成功、實際上不能用」的失敗模式，會讓你對「Publish 成功」這件事的信任打多少折扣？除了逐一檢查 \$metadata 之外，還有什麼方法可以更早發現這類問題？
5. 這一課的 Adapter 驗證程式意外發現 /ui2/cl_json=>deserialize 對某些格式錯誤的 JSON 不會拋例外，而是安靜地留下空結構——如果你要幫這個 Adapter 補上更嚴謹的輸入驗證，你會檢查哪些欄位、用什麼方式判斷「這筆請求資料是不是真的有效」？

答案

見 zrap10_log.tabl.abap、zi_rap10_log.ddls.abap、zi_rap10_log.bdef.abap、
zbp_i_rap10_log.clas.abap、zbp_i_rap10_log.clas.locals_imp.abap、
zr_rap10_demo.prog.abap、zrap10_sd.srvd.abap、zcl_rap10_rest_handler.clas.abap、
zcl_rap10_resource.clas.abap、zr_rap10_adapter_demo.prog.abap。反面教材：
za_rap10_hdr.ddls.abap、za_rap10_dtl.ddls.abap、za_rap10_hdr.bdef.abap（檔頭有完整錯誤訊息
記錄）。SAP 端物件：以上皆為 \$TMP 套件（ZRAP10_SB 為使用者於 Eclipse 建立）。